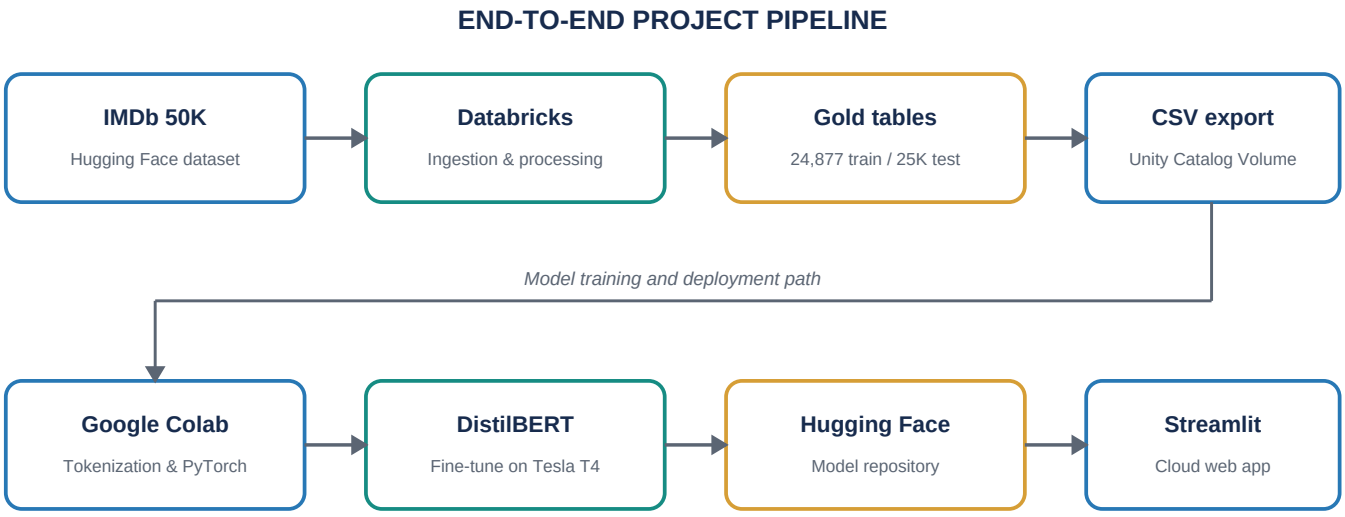
IMDb SENTIMENT ANALYSIS PROJECT PIPELINE

From dataset ingestion to deployed sentiment prediction



Pipeline outcome: a Databricks-prepared IMDb dataset was used to fine-tune DistilBERT in Google Colab. The saved model was published to Hugging Face and served through a Streamlit Community Cloud application.

1. Pipeline at a glance

Stage	Platform / tool	Main output
Dataset source	Hugging Face Datasets — stanfordnlp/imdb	25,000 train + 25,000 test labelled reviews
Data engineering	Databricks Spark 4.2.0	Bronze, Silver and Gold tables
Data export	Unity Catalog Volume → CSV	24,877 train + 25,000 test rows
Model development	Google Colab, PyTorch, Transformers	Fine-tuned DistilBERT model
Model hosting	Hugging Face Model Hub	Model weights + tokenizer + config
Application	Streamlit Community Cloud	Interactive review sentiment prediction

2. Data engineering pipeline — Databricks

The data engineering stage used the Medallion Architecture to separate raw ingestion, text standardization and model-ready dataset preparation.

2.1 Bronze layer — ingestion

Source: Hugging Face dataset **stanfordnlp/imdb**. The labelled train and test splits were ingested into the Databricks catalog **imdb_sentiment**.

Table	Rows	Contents
bronze.train_reviews	25,000	Original review text, label, ingested_at
bronze.test_reviews	25,000	Original review text, label, ingested_at

Purpose: preserve the ingested source data and establish a starting point for downstream transformations.

2.2 Silver layer — cleaning and validation

Transformations applied:

Operation	Purpose / result
HTML line-break cleanup	Replaced HTML line-break tags such as with spaces.
Whitespace trimming	Removed leading and trailing whitespace.
Derived fields	Created clean_text and text_length.
Quality checks	Checked nulls, empty reviews, invalid labels and remaining line-break markup.

Silver output: 25,000 train rows and 25,000 test rows. Validation recorded zero nulls, zero empty reviews, zero invalid labels and zero remaining line-break tags.

2.3 Gold layer — leakage-aware split preparation

A comparison of distinct clean_text values found 123 exact review-text overlaps between the original training and test splits. All 123 pairs had matching labels: 95 negative and 28 positive. To avoid exact duplicate texts crossing the train/test boundary, the overlapping records were removed from training only, using a left anti join. The test split was retained unchanged.

Gold table	Rows	Label distribution
gold.train_reviews	24,877	12,405 negative; 12,472 positive
gold.test_reviews	25,000	12,500 negative; 12,500 positive

Post-check: zero shared unique review texts between Gold train and Gold test.

3. Export and handoff to Google Colab

Gold tables were exported through a Unity Catalog Volume because the Databricks Free Edition public DBFS root was disabled. The workflow initially attempted Parquet, but Colab encountered Arrow parsing errors. The final handoff used CSV files with UTF-8 encoding and explicit quote escaping.

Export artifact	Purpose
train_v2.csv	Gold training records (24,877 rows)
test_v2.csv	Gold test records (25,000 rows)

In Colab, the CSVs loaded successfully into Pandas. Both datasets had three columns: clean_text, label and text_length. Null and empty-text checks returned zero; labels were restricted to 0 and 1.

4. NLP preprocessing and input pipeline

The raw review text was passed to the pretrained DistilBERT tokenizer. No manual token-ID creation or aggressive stopword removal was needed.

Step	Implementation	Reason
Tokenizer	distilbert/distilbert-base-uncased	Convert text into WordPiece tokens and model inputs
Sequence limit	MAX_LENGTH = 256; truncation enabled	Bound memory and compute for free GPU training
Padding	Dynamic padding with DataCollatorWithPadding	Pad each batch to its longest sequence
Dataset wrapper	PyTorch Dataset	Return input_ids, attention_mask and labels as tensors
Batching	DataLoader; batch size 8	Feed manageable batches to the GPU

Important trade-off: EDA showed that 44.25% of reviews exceeded 256 tokens. Those reviews were truncated to the first 256 tokens in this experiment. This was a compute-conscious baseline, not proof that 256 tokens is the optimal length.

5. Model fine-tuning — Google Colab

The model was loaded from the base DistilBERT checkpoint with a two-class sequence-classification head. The classifier head was newly initialized and learned during fine-tuning.

Training parameter	Value
Base model	distilbert/distilbert-base-uncased
Number of labels	2 (Negative = 0, Positive = 1)
Train / test rows	24,877 / 25,000
Epochs	2
Batch size	8
Gradient accumulation	2 (effective batch size 16)
Learning rate	2e-5
Weight decay	0.01
Optimizer	AdamW
Scheduler	Linear warmup and decay; 10% warmup
Precision / hardware	FP16 mixed precision / NVIDIA Tesla T4

Training loss by epoch

Epoch	Training loss	Approx. time
1	0.3014	3 min 10 sec
2	0.1534	3 min 11 sec

Training loss decreased by approximately 49.1% between epoch 1 and epoch 2. Epoch checkpoints were saved, and the final model and tokenizer were saved separately for inference.

6. Evaluation pipeline

After training, the model was switched to evaluation mode. Test batches were passed through the model without gradient calculation. The highest logit determined the predicted class; predictions were compared with the test labels.

Metric	Result
Test loss	0.2213
Accuracy	91.69%
Positive precision	90.72%
Positive recall	92.87%
Positive F1-score	91.79%

Confusion matrix

Actual / predicted	Predicted negative	Predicted positive
Actual negative	11,313 (TN)	1,187 (FP)
Actual positive	891 (FN)	11,609 (TP)

The model correctly classified 22,922 reviews and incorrectly classified 2,078. False positives were 1,187 and false negatives were 891.

7. Model packaging and Hugging Face hosting

The final model and tokenizer were saved using the Hugging Face save_pretrained workflow. The local final model folder contained the following four essential files:

File	Role
config.json	Model architecture and configuration
model.safetensors	Fine-tuned model weights (approximately 255 MB)
tokenizer.json	Tokenizer vocabulary and tokenization data
tokenizer_config.json	Tokenizer configuration

The model was uploaded to the public Hugging Face repository: SHAILESHGOKU/imdb-distilbert-sentiment. This kept the large model file out of GitHub.

8. Inference and application deployment

A prediction function accepts raw review text, tokenizes it, moves tensors to the same device as the model, performs inference and applies softmax to the two logits. The predicted sentiment is the class with the larger probability.

Custom test	Prediction	Confidence
Clearly positive review	Positive	99.66%
Clearly negative review	Negative	99.56%

A CPU/GPU device mismatch was encountered during local inference and resolved by explicitly moving the model to cuda:0 and ensuring input tensors used the same device. Custom examples verified the inference path; they are not a substitute for test-set evaluation.

Deployment architecture

Component	Role
GitHub	Stores app.py, requirements.txt and README.md
Hugging Face	Stores the model and tokenizer files; application downloads them at startup

Component	Role
Streamlit Community Cloud	Runs the browser-based sentiment analysis application

The project owner confirmed successful deployment to Streamlit Community Cloud. The exact live application URL was not included in the project details available for this document.

9. End-to-end operational flow

No.	Pipeline activity	Result / handoff
1	Load IMDb dataset	25K train + 25K test source records
2	Ingest into Databricks Bronze	Raw train/test tables
3	Clean and validate in Silver	clean_text, text_length; quality checks passed
4	Remove train/test exact overlap in Gold	24,877 train; 25,000 test; zero shared unique texts
5	Export Gold data to CSV	train_v2.csv and test_v2.csv
6	Load CSVs in Colab	Pandas dataframes validated
7	Tokenize and build PyTorch datasets	Dynamic padding; sequence limit 256
8	Fine-tune DistilBERT	2 epochs on Tesla T4; final model saved
9	Evaluate on held-out test set	91.69% accuracy; 91.79% positive F1
10	Upload model to Hugging Face	Public model repository
11	Build and deploy Streamlit app	Successful deployment confirmed

10. Key implementation decisions

Decision	Reasoning and trade-off
Databricks Medallion Architecture	Separated ingestion, standardization and model-ready data; made each stage easier to validate.
Remove exact overlap from train only	Avoided exact duplicate review text crossing the train/test boundary while retaining the full test evaluation set.
CSV instead of Parquet for handoff	CSV with explicit quote escaping loaded successfully in Colab after Parquet parsing problems.
256-token baseline	Reduced compute requirements on free Colab GPU, at the cost of truncating a substantial share of long reviews.
Dynamic padding	Avoided padding every review to the maximum length in every batch.
Hugging Face for model hosting	Kept the 255 MB model weights outside GitHub and made the model retrievable by the app.
CPU inference in Streamlit	Avoided depending on a GPU in the hosted environment.

11. Limitations and next engineering steps

The following items are possible improvements, not completed pipeline stages:

- Compare sequence limits of 256, 384 and 512 with a validation split from training.
- Keep the current test set reserved for final evaluation rather than repeated tuning.
- Add a warning in the app when the review is truncated.
- Perform larger, structured false-positive and false-negative analysis.
- Pin compatible package versions and document model/data versions.
- Monitor hosted startup time and inference latency.

12. Pipeline conclusion

The completed pipeline moves from a public IMDb dataset through Databricks data engineering, Colab-based DistilBERT fine-tuning, Hugging Face model hosting and Streamlit application deployment. It produced 24,877 leakage-checked training records and 25,000 held-out test records, achieved 91.69% test accuracy, and delivered a working browser-based sentiment prediction application.

Report scope note

This pipeline document is based on the implementation details and outputs recorded during the project workflow. It does not independently rerun the notebooks or inspect the current cloud deployment. The exact GitHub repository URL and Streamlit application URL were not supplied and are therefore not invented.